Project Title:

Card Matching Game — “Flip & Match: Duel Edition”

1. Problem Statement

Most console-based matching games are for single players and lack competition or time challenges. Our goal is to create a two-player, turn-based memory game that adds real-time thinking, competition, and fun, while applying object-oriented programming and data structure concepts.

2. Proposed Solution

We will design a two-player, turn-based memory game where players take turns flipping two cards to find pairs. Each card stays visible for a short moment before flipping back if not matched. The system will use OOP and DSA principles for clean structure and smooth data handling. The game will also include a timer per turn, score tracking, and winner display.

3. Gameplay Rules

• All cards start face down. • Both players get 5 seconds to view the entire board at the start. • Each player has 10 seconds per turn: - If no card is flipped, the turn automatically passes. - If only one card is flipped when time ends, it flips back and the turn passes. - If two cards are flipped: → If matched: they stay face up, score +1, and control switches. → If not matched: both flip back after a pause, then switch turns. • Game ends when all pairs are matched — higher score wins, equal scores mean a draw.

4. System Design / Classes

Card<T>: Template for storing card symbol/value. Uses overloaded == operator.
Board: Manages the 2D grid of cards using linked lists.
Player: Stores name, score, and timer.
Game: Main controller that handles turns, timing, and scoring.

5. Game Logic

1. Initialize and shuffle cards into a 2D grid.
2. Reveal all cards for 5 seconds, then hide them.
3. Player 1 starts and has 10 seconds per turn.
4. Player flips two cards — if matched, score increases; if not, they flip back.
5. Timeout automatically switches turns.
6. Continue until all pairs are matched, then display winner or draw.

6. Data Structures (Technical Implementation)

Linked List: Dynamic storage and shuffling of cards.
Stack: Stores previous moves for undo.
Hash Table: Tracks matched cards for O(1) lookup.
Timer: Manages turns fairly.
Array (2D): Represents the game board.

7. OOP Concepts Used

Encapsulation: Player data kept private.
Abstraction: Game functions hide inner workings.
Inheritance: Allows extensions like TimedGame.
Polymorphism: Overloaded operators and flexible behaviors.
Templates: Generic card types for flexibility.

8. Tools & Technologies

Language: C++
IDE: Code::Blocks / Replit
Version Control: GitHub
Optional Web: React.js, TailwindCSS
Deployment: Netlify / Replit

9. Web Extension (Optional)

If developed as a web app, React will manage card animations and timers. C++ logic can be adapted in TypeScript. Timers will use setTimeout() and setInterval(). Game can be hosted on Netlify or Replit for multiplayer play.

10. Expected Outcome

A fully working two-player memory game with scoring and timer features. It will show solid OOP and data structure use, with an optional online version.

11. Team Members & Roles

Taha	Game Logic & Timer
Ahmad	Data Structures & Debugging
Shujaan	UI Design & Documentation

12. Timeline (Gantt Overview)

Week 1: Planning & Class Design
Week 2: Card, Player, Board Implementation
Week 3: Game Logic & Timer
Week 4: Debugging & Scoring Tests
Week 5: Web Adaptation
Week 6: Final Presentation

13. Future Enhancements

• Add sound and animations.
• Online multiplayer with sockets.
• Difficulty levels (board size, timer).
• Save/load progress feature.